

Complete Gateway Architecture

AI-Based SQL Injection & XSS Detection Gateway

This document specifies the full architecture: a security gateway whose detection core is a machine-learning classifier rather than static regex rules, with the supporting infrastructure (session tracking, risk scoring, policy enforcement, incident memory, and analytics) that any real gateway needs regardless of what powers detection.

Pipeline Order

1. Request Ingress
2. Parser
3. Preprocessing
4. ML Detection Engine
5. Threat Intelligence
6. Session Manager
7. Device Profiler
8. Behavior Engine
9. Risk Engine
10. Decision Engine
11. Policy Engine
12. Incident & Memory Service
13. Analytics & Reporting
14. Dashboard (SOC UI)

Component Responsibilities

#	Component	Responsibility
1	Request Ingress	Accept traffic as a reverse proxy or middleware in front of the protected app
2	Parser	Normalize raw request data into a consistent internal object
3	Preprocessing	Decode encoded characters, normalize case, tokenize every input field
4	ML Detection Engine	Classify each field as benign / SQLi / XSS with a confidence score
5	Threat Intelligence	Enrich with IP reputation, GeoIP/ASN, known-malicious history
6	Session Manager	Maintain per-session state: request counts, visited endpoints, duration
7	Device Profiler	Fingerprint the client device from header combinations
8	Behavior Engine	Score anomalies: request rate, endpoint enumeration, session length
9	Risk Engine	Fuse ML confidence, anomalies, and reputation into one capped score

10	Decision Engine	Convert the risk score into ALLOW / MONITOR / RATE_LIMIT / BLOCK
11	Policy Engine	Apply hard overrides for critical attack types regardless of score
12	Incident & Memory Service	Persist flagged events; update long-term attacker memory by IP/device
13	Analytics & Reporting	Aggregate incidents into trends, breakdowns, and exportable reports
14	Dashboard (SOC UI)	Let analysts review incidents, replay timelines, confirm or override

Dependencies

Component	Depends on
Parser	Live request object (Flask request)
Preprocessing	Parser output
ML Detection Engine	Trained PyTorch model + tokenizer/vocab artifacts (<code>model.pt</code>)
Threat Intelligence	IP reputation store, GeoIP/ASN lookup service
Session Manager	Server-side in-memory session store (Flask-Caching / SimpleCache), session cookie
Device Profiler	Header fields from Parser, hashing (SHA-256)
Behavior Engine	Context fields from Session Manager, Device Profiler, ML Detection Engine
Risk Engine	Risk weight config, outputs of stages 4–8
Decision Engine	Decision thresholds, Risk Engine output
Policy Engine	Attack-type list, Decision Engine output
Incident & Memory Service	Primary data store (MongoDB/PostgreSQL)
Analytics & Reporting	Incident & Memory Service
Dashboard	Analytics, Incident, and Reporting APIs; local UI cache (node-cache)

Input / Output

Input (per request): HTTP method, path, query parameters, headers, cookies, body, client IP, user agent.

Output (per request): - Decision: ALLOW / MONITOR / RATE_LIMIT / BLOCK - Attack type(s) detected (if any): SQLI, XSS, or NONE - ML confidence score (0–1) - Composite risk score (0–100, capped) - Decision reason (which stage triggered the verdict) - Persisted incident record (if flagged)

Detailed Component Notes

4. ML Detection Engine — the core differentiator

- **PyTorch, CUDA-enabled build:** a character-level CNN or BiLSTM trained directly on raw/tokenized payload text, capturing sequential and structural patterns in SQLi/XSS payloads — this is the sole detector, with both training and inference running on GPU
- No scikit-learn or classical-model baseline — PyTorch is the entire detection path, end to end
- Outputs a confidence score, not just a hard label — this score feeds directly into the Risk Engine (stage 9), rather than being a binary trigger like a regex match

- Model artifacts are versioned (PyTorch `.pt` checkpoint + tokenizer/vocab) so a rollback is possible if a newly trained model regresses
- Verify GPU is actually in use with `torch.cuda.is_available()` before training/serving — a CPU-only PyTorch install will silently fall back and lose the performance benefit

6. Session Manager — server-side caching (security-critical, not swappable to the client)

- Uses an in-memory cache on the Flask side (Flask-Caching with the SimpleCache backend) instead of a dedicated cache server like Redis
- This state **must stay server-side**: it tracks the request sender's own behavior (request counts, rate-limit counters) to catch repeated/escalating attacks. If this lived in client-side storage, an attacker could simply clear it and reset their own rate limit — it has to be authoritative and out of the client's control
- Trade-off: SimpleCache is per-process, so state is only consistent if the gateway runs as a single Flask process; acceptable for a localhost/single-instance deployment at this scale (not expected to exceed ~1,000 users)

8. Behavior Engine — anomaly rules

Rule	Condition	Score contribution
High request rate	<code>request_count > 100</code>	+20
New/unknown device	Device unseen before	+10
Multiple attack types	2+ types from same source	+20
Long session	<code>Session > 3600s</code>	+10
Endpoint enumeration	> 20 unique endpoints visited	+20
Known malicious IP	Prior confirmed attacks on record	+30
Attack detected	Any ML-flagged attack this request	+20

9-10. Risk Engine & Decision Engine

- Risk score = weighted sum of ML confidence, behavior score, and reputation signals, capped at 100
- Score bands map to decisions: low → ALLOW, medium → MONITOR, high → RATE_LIMIT, critical → BLOCK
- Known-malicious IPs are always BLOCK regardless of score (conservative default)

11. Policy Engine — overrides

Attack Type	Forced Result
SQL Injection (high confidence)	BLOCK
XSS (high confidence)	RATE_LIMIT
SQL Injection + repeat offender	BLOCK

14. Dashboard (SOC UI) — client-side caching

- The React dashboard runs its own local Node process (localhost), so **node-cache** is used there for UI-level convenience caching — last-fetched incident lists, open tab/filter state, anything that just makes the dashboard feel responsive
- This is separate and unrelated to the Session Manager (stage 6): the dashboard cache is for the analyst's browsing experience, not for tracking or scoring attacker behavior, so it's fine for it to be lightweight and local
- At this scale (localhost, not expected to exceed ~1,000 users), no shared/distributed cache is needed for either side

Request Lifecycle (Narrative)

1. A request arrives at the gateway (reverse proxy or middleware mode)
2. Parser builds a normalized request context
3. Preprocessing cleans and tokenizes every input field
4. The ML Detection Engine scores each field for SQLi/XSS likelihood
5. Threat Intelligence, Session Manager, and Device Profiler enrich the context in parallel
6. Behavior Engine scores anomalies using the enriched context
7. Risk Engine fuses all signals into one capped score
8. Decision Engine converts the score into a verdict
9. Policy Engine applies any hard overrides
10. If flagged, Incident & Memory Service persists the event and updates attacker memory
11. If allowed, the request is forwarded to the upstream application; otherwise a block/rate-limit response is returned
12. Incidents surface on the Dashboard for analyst review

Deployment Modes

- **Reverse Proxy Mode:** the gateway sits in front of the real application and forwards allowed traffic upstream, similar to a WAF appliance
- **Middleware Mode:** the application calls the gateway's decision API before proceeding with each request, and respects the returned verdict (fail-closed recommended — reject traffic if the gateway is unreachable, rather than failing open)
- Both modes run directly on the host (Flask process + local Node process for the dashboard) — no containerization required at this scale

Data Store Design (Collections/Tables)

Store	Purpose
incidents	Every flagged request: verdict, risk score, attack type, timestamp, context
sessions	Per-session state: request counts, endpoints visited, duration
devices	Device fingerprints, first/last seen, request counts
security_memory	Long-term attacker memory keyed by IP/device, attack history
model_versions	Trained model metadata: version, training date, validation metrics

Tech Stack

Layer	Technology
Gateway runtime	Flask (Python)
ML	PyTorch, CUDA-enabled build (GPU-accelerated char-level CNN/BiLSTM)
Gateway-side session cache (server, security-critical)	Flask-Caching (SimpleCache, in-memory)
Dashboard-side UI cache (client, convenience only)	node-cache (local Node process on localhost)
Primary data store	MongoDB or PostgreSQL

Dashboard	React
Deployment	Reverse proxy or middleware, run directly on host

Why This Differs From a Regex-Based Gateway

A signature/regex-based scanner can only catch attack patterns someone has already anticipated, and it never improves without a human rewriting rules. This architecture keeps every piece of infrastructure a real gateway needs — session tracking, device profiling, risk scoring, policy enforcement, incident memory, analytics, and a SOC dashboard — but replaces the static detection core with a trained PyTorch classifier that generalizes to payload variants it wasn't explicitly given as a rule, rather than matching only what a regex author anticipated in advance.